SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. *(BL3)*

Assessment Tool Weightage: 10%

Duration :60 Mins
On Class
QUESTIONS: 1 **Total Marks:50**

Annexure A

TECHNICAL PRESENTATION

Code of Conduct:

I __**E.Kavya(192424365)_**__ *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.
E.Kavya
Signature of the student:

Annexure A

TECHNICAL PRESENTATION

Q1. Phases of a Compiler and Their Role

Understanding of Case

a) Explains all compiler phases such as lexical, syntax, semantic, intermediate, optimization, and code generation.
b) Describes the role of each phase in transforming source code into machine code.

Application of Concepts

c) Applies compiler phases to detect errors like invalid tokens, missing syntax, and semantic mismatches.
d) Demonstrates phase-wise error identification using real C program examples.

Analysis

e) Analyzes how each phase depends on the previous phase for correct execution.
f) Evaluates how early-stage errors (lexical/syntax) stop further compilation.

Solution Design

g) Suggests integrating better error recovery techniques across phases.
h) Proposes modular compiler design for flexibility and easier debugging.

Clarity

i) Uses flowcharts to explain phase sequence clearly.
j) Provides step-by-step examples showing how code is processed.

Q2. Lexical Analysis and Token Specification using LEX

Understanding of Case

a) Defines tokens, lexemes, and patterns used in lexical analysis.
b) Explains the role of the lexical analyzer in scanning source code.

Application of Concepts

c) Demonstrates token generation using LEX tool with sample inputs.
d) Applies regular expressions to identify identifiers, constants, and operators.

Analysis

e) Analyzes common lexical errors such as invalid identifiers and illegal symbols.

f) Evaluates how tokenization improves parsing efficiency.

Solution Design

g) Designs token rules using regular expressions in LEX.
h) Implements error-handling mechanisms in lexical analysis.

Clarity

i) Provides clear LEX syntax examples for better understanding.
j) Uses input-output illustrations for token generation.

Q3. Language Processors: Compiler vs Interpreter

Understanding of Case

a) Differentiates between compiler and interpreter based on execution method.
b) Explains how each processor converts high-level code.

Application of Concepts

c) Applies concepts to languages like C (compiler) and Python (interpreter).
d) Demonstrates how errors are handled differently in both.

Analysis

e) Compares performance, speed, and memory usage of both processors.
f) Evaluates advantages and disadvantages in real-world scenarios.

Solution Design

g) Suggests using compilers for performance-critical applications.
h) Recommends interpreters for rapid development and debugging.

Clarity

i) Uses comparison tables for easy understanding.
j) Includes real-life examples and execution flow diagrams.

Q4. Grouping of Compiler Phases and Optimization

Understanding of Case

a) Explains grouping into front-end (analysis) and back-end (synthesis).
b) Describes the role of each group in compilation.

Application of Concepts

c) Applies grouping to simplify compiler design and debugging.

d) Demonstrates how front-end handles errors and back-end improves performance.

Analysis

e) Analyzes how grouping improves modularity and maintainability.

f) Evaluates optimization techniques like constant folding and dead code removal.

Solution Design

g) Designs efficient compiler structure using grouped phases.

h) Suggests advanced optimization techniques for better performance.

Clarity

i) Uses diagrams to represent front-end and back-end clearly.

j) Provides before-and-after code examples for optimization.

Q5. Compiler Construction Tools and LEX Understanding of Concepts

a) Explain compiler construction tools.

b) Describe the structure of a LEX program.

Application of Concepts

c) Apply LEX to generate tokens for a simple program.

d) Demonstrate writing rules for identifiers and constants.

Analysis

e) Analyze advantages of using LEX in compiler design.

f) Evaluate limitations of LEX.

Solution / Design

g) Design a simple lexical analyzer using LEX rules.

Clarity

h) Show LEX workflow (input → scanner → tokens).

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-dept analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

SIMATS ENGINEERING
Engineer to Excel

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Grouping of Compiler Phases and Optimization

Technical Presentation

Guided by:
Dr.G.Michael

Name:E.Kavya
Reg No:192424365

Understanding of Case
a) Grouping of Compiler Phases

Compiler phases are divided into two major groups:

- Front End
- Back End

The Front End analyzes the source program, while the Back End generates optimized machine code for execution.

b) Describes the role of each groupincompilation.
Front End

The Front End is responsible for analyzing the source code and checking whether it follows the rules of the programming language.

Purpose:

- Detects errors in the source program.
- Analyzes the structure and meaning of the code.
- Produces machine-independent intermediate code.

Phases Included:

- Lexical Analysis
- Syntax Analysis
- Semantic Analysis
- Intermediate Code Generation

BackEnd

- The Back End converts the intermediate code into optimized machine code for the target system.

Purpose:

- Improves program performance.
- Reduces execution time and memory usage.
- Produces efficient machine code.

Phases Included:

- Code Optimization
- Code Generation

Applications
c)Application of Grouped Compiler Phases

- Operating Systems
- Mobile Applications
- Embedded Systems
- Artificial Intelligence
- Database Systems
- Cloud Computing
- Robotics
- Game Development

Advantages and Disadvantages
d)Analysis of Grouped Compiler Design

Advantages	Disadvantages
Faster program execution.	Increases compiler complexity.
Reduces memory usage.	Takes more compilation time.
Generates efficient machine code.	Requires more compiler resources.
Detects errors early.	Optimized code is harder to debug.
Improves overall performance.	Some optimizations depend on the target machine.

Solution Design

e)Design of grouped compiler structure

Steps:

1.Combine lexical, syntax, semantic → Front End

2.Generate intermediate code

3.Optimize intermediate code

4.Convert into target machine code

f)Example flow:

Input:

- int a = b + 5;

Processing:

- Front End → checks syntax & meaning
- Intermediate Code →t1 = b + 5,a = t1
- Back End → machine instructions

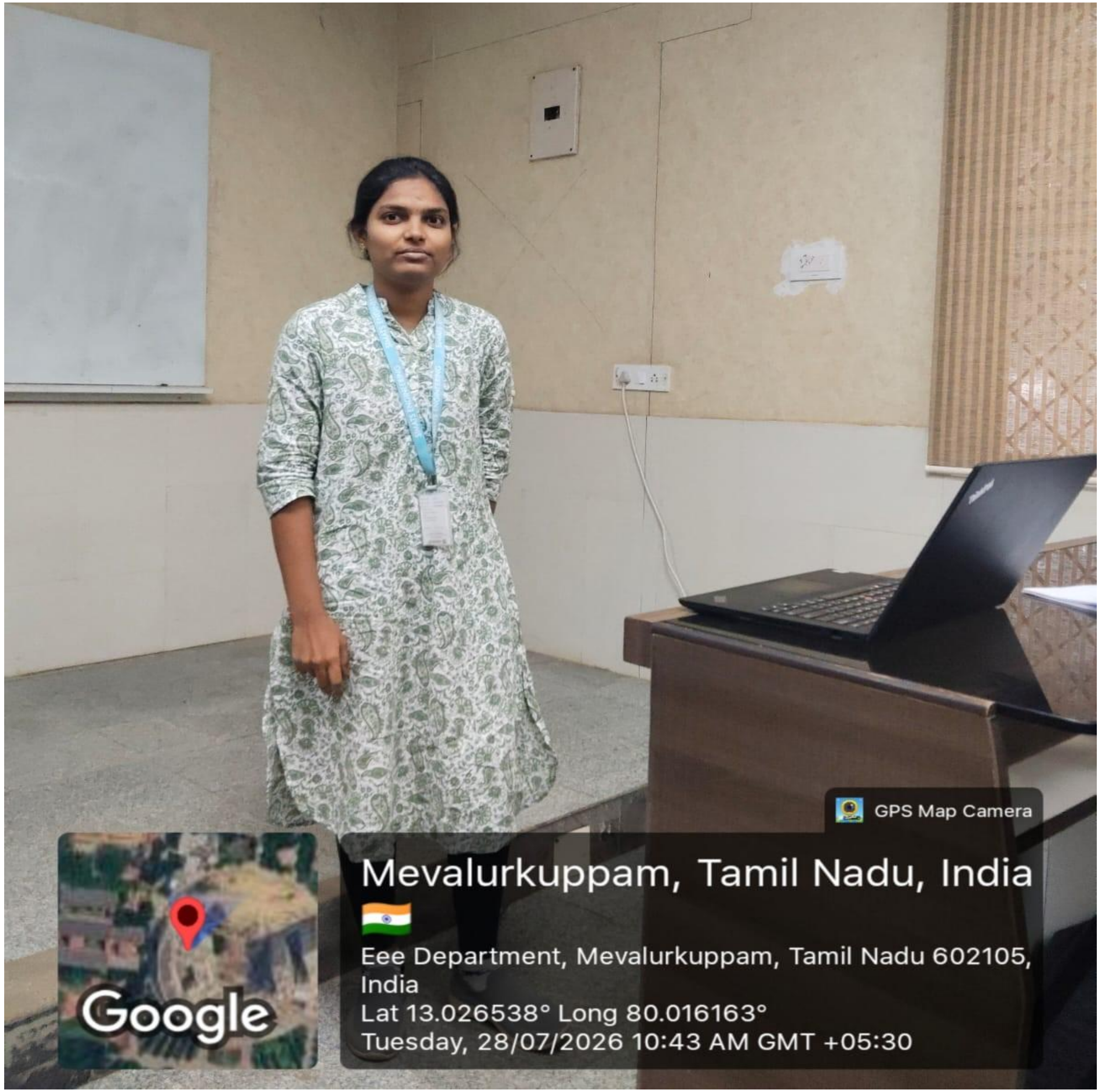
Output:

- Executable machine code

Conclusion

- Compiler phases are grouped into Front End and Back End.
- The Front End checks the source code and finds errors.
- The Back End optimizes the code and generates machine code.
- Code optimization improves speed and reduces memory usage.
- Grouping the compiler phases makes the compilation process simple and efficient.

THANK YOU


GPS Map Camera
Mevalurkuppam, Tamil Nadu, India
Eee Department, Mevalurkuppam, Tamil Nadu 602105, India
Lat 13.026538° Long 80.016163°
Tuesday, 28/07/2026 10:43 AM GMT +05:30
Google